Memory Efficient Doubly Linked List – Insert At END And Time Complexity.

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    while (curr != nullptr)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr){ break; }
        prev = curr;
        curr = next;
    }
    curr->npx = XOR(prev, newNode);
    newNode->npx = XOR(curr, nullptr);
    cout << data << " inserted at the end." << endl;
}
……
      case 2:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtEnd(data);
            break;
```

Program Analysis:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ ***The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.***

→ ***The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

→ ***data(an integer) is typically 4 bytes.***

→ ***npx(a pointer to another Node) is typically 8 bytes.***

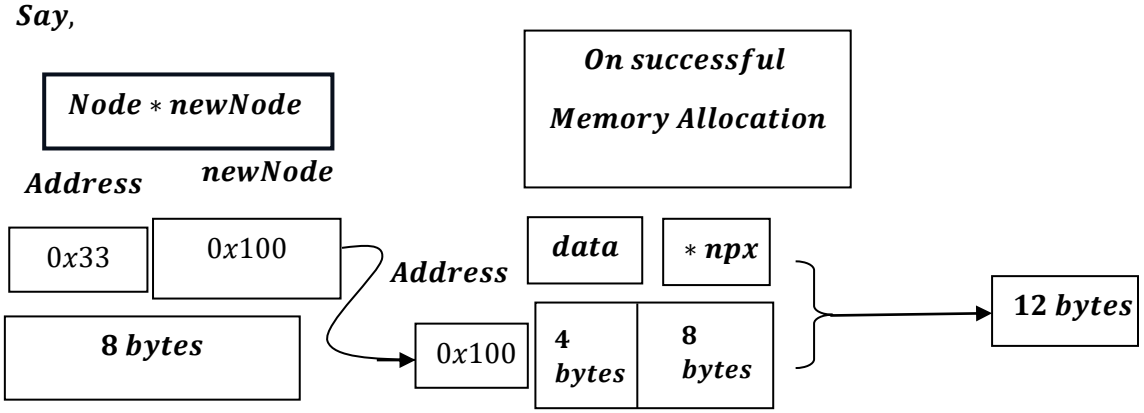
→ ***Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx)*** .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

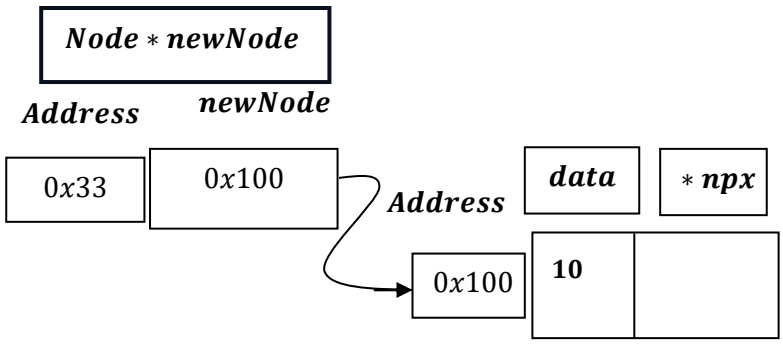
→ ***newNode will try to allocate memory for data i.e. 4 bytes , npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

Say,



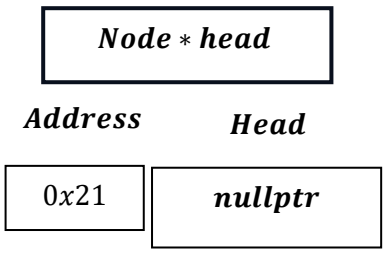
Lets say , there is data → 10 , to be entered.

```
    newNode->data = data;
```



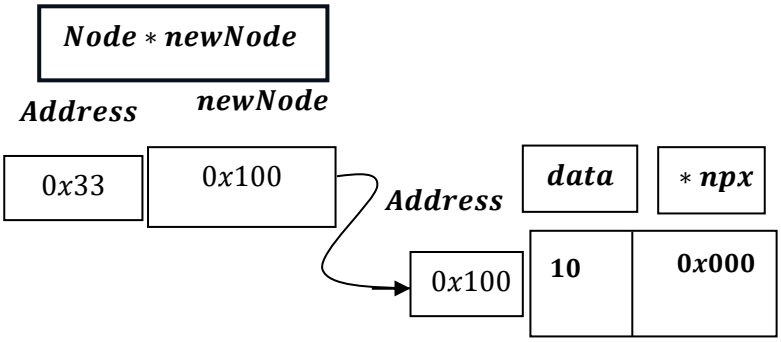
When List is Empty

```
    if (head == nullptr)
    {
        newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
```



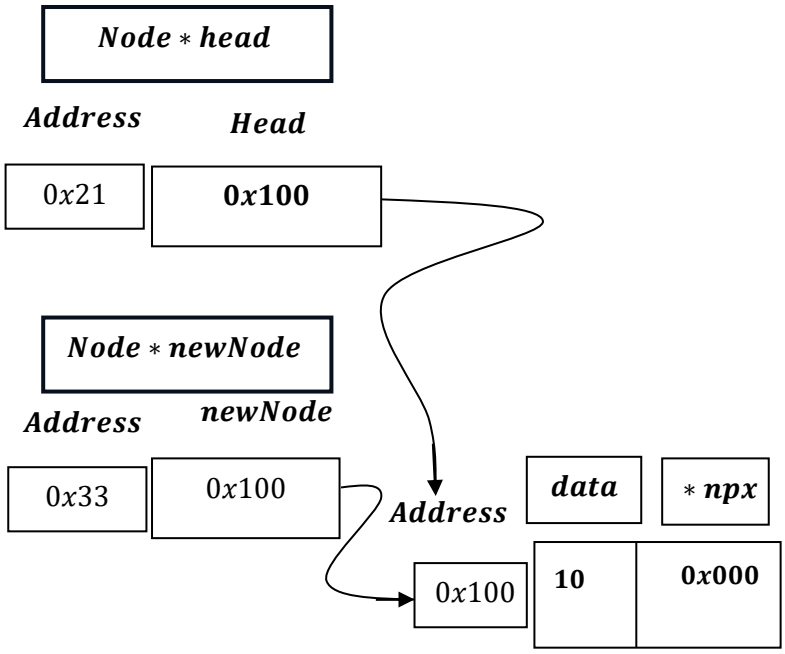

```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
newNode->npx = XOR(nullptr, nullptr); // NULL ^ NULL
```

$\boldsymbol{newNode \rightarrow npx = XOR(NULL: 0x000\ , NULL: 0x000) \Longrightarrow 0x000 \oplus 0x000 = 0x000.}$



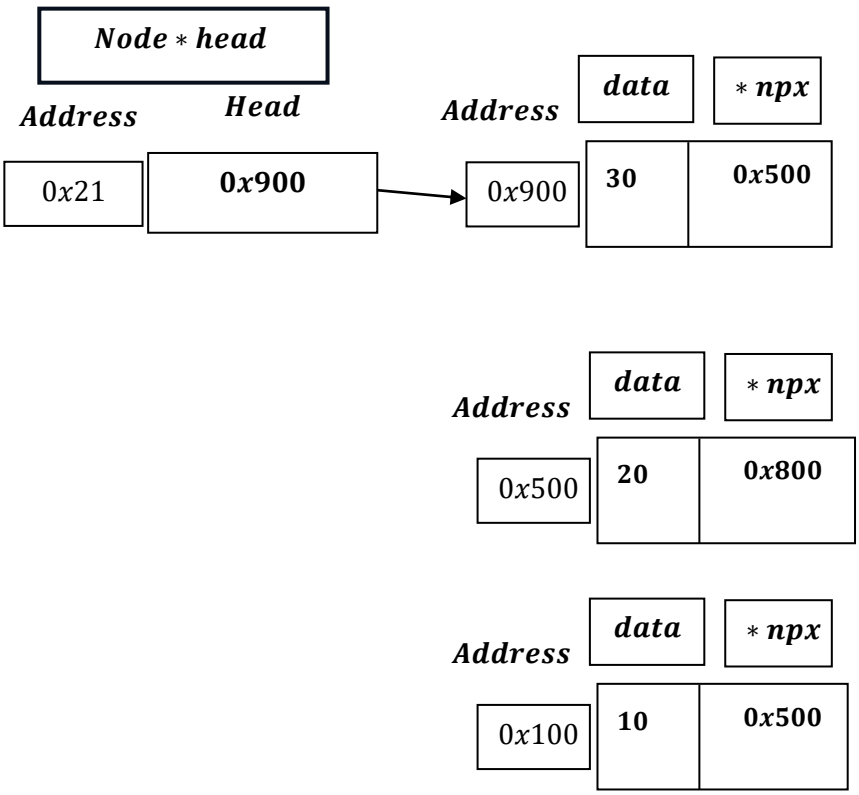

```
        head = newNode;
```

$\boldsymbol{head = newNode = 0x100.}$



And, it prints*: ``10 *inserted at end.*`` *and Return end the if block and exit from the function.

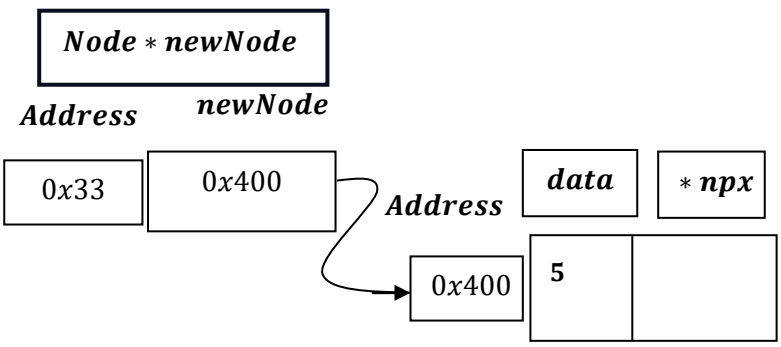
When List is not Empty



And,

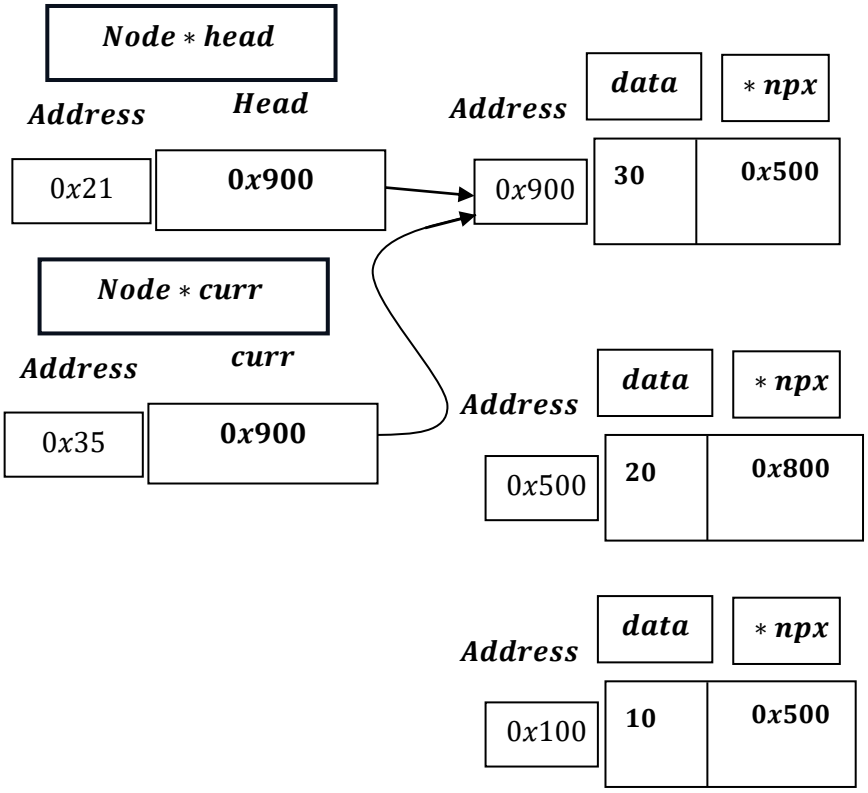
Lets say , there is data → 5 , to be entered.

```
    newNode->data = data;
```



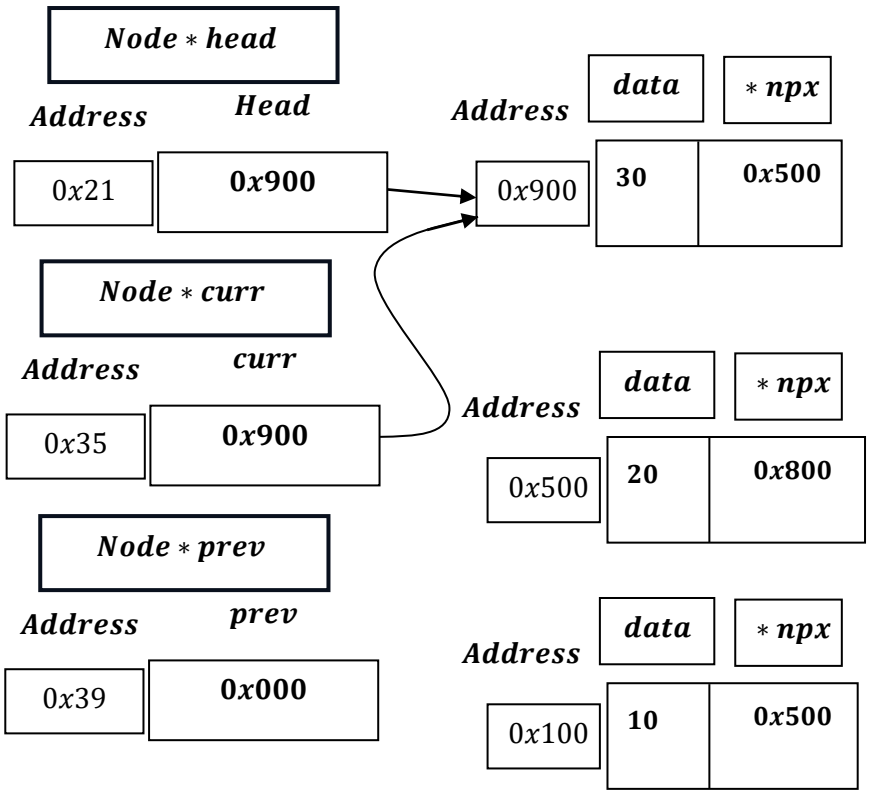
```
    Node *curr = head;
```

$\boldsymbol{Node * curr = head = 0x900}$.

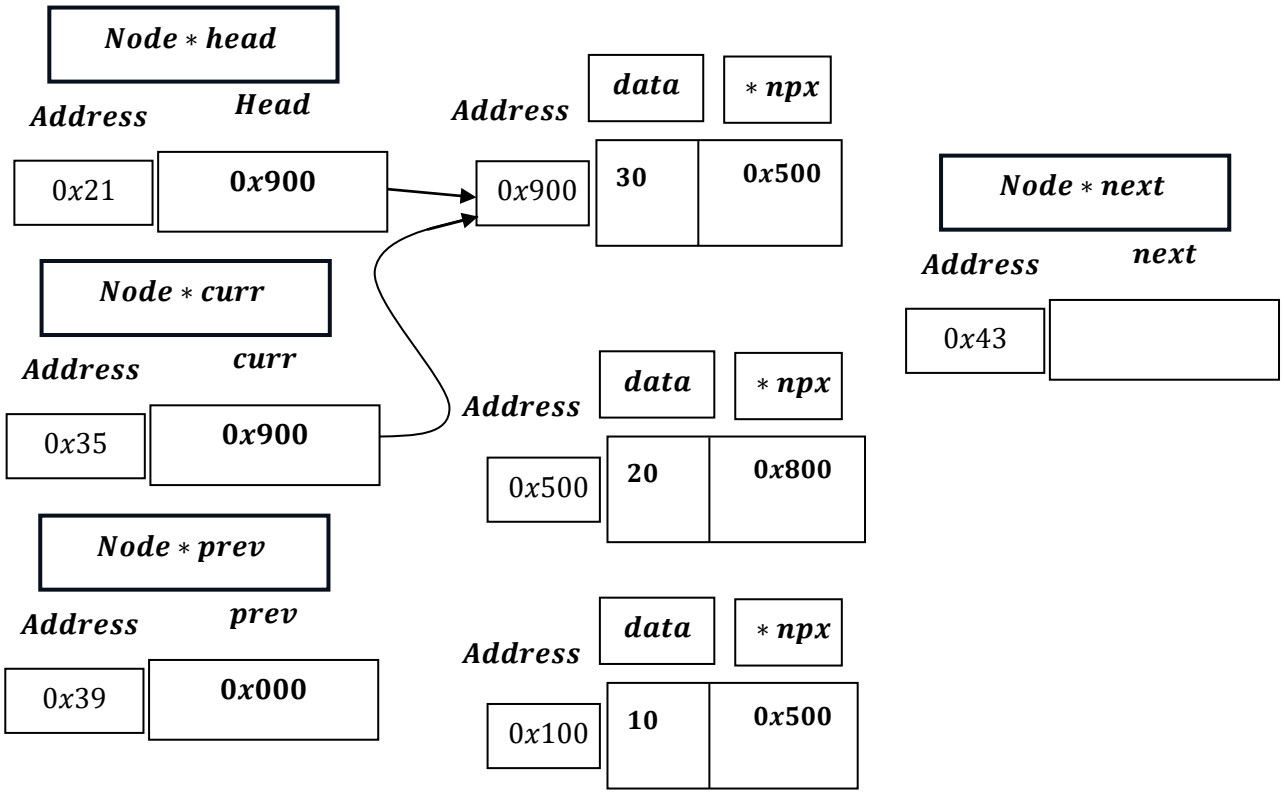



```
    Node *prev = nullptr;
```

$\boldsymbol{Node * prev = nullptr = 0x000}$;


Node * head
Address
Head
0x21
0x900
Node * curr
Address
curr
0x35
0x900
Node * prev
Address
prev
0x39
0x000
Address
data
* npx
0x900
30
0x500
Address
data
* npx
0x500
20
0x800
Address
data
* npx
0x100
10
0x500

```
    Node *next;
```


Node * head
Address
Head
0x21
0x900
Node * curr
Address
curr
0x35
0x900
Node * prev
Address
prev
0x39
0x000
Address
data
* npx
0x900
30
0x500
Address
data
* npx
0x500
20
0x800
Address
data
* npx
0x100
10
0x500
Node * next
Address
next
0x43

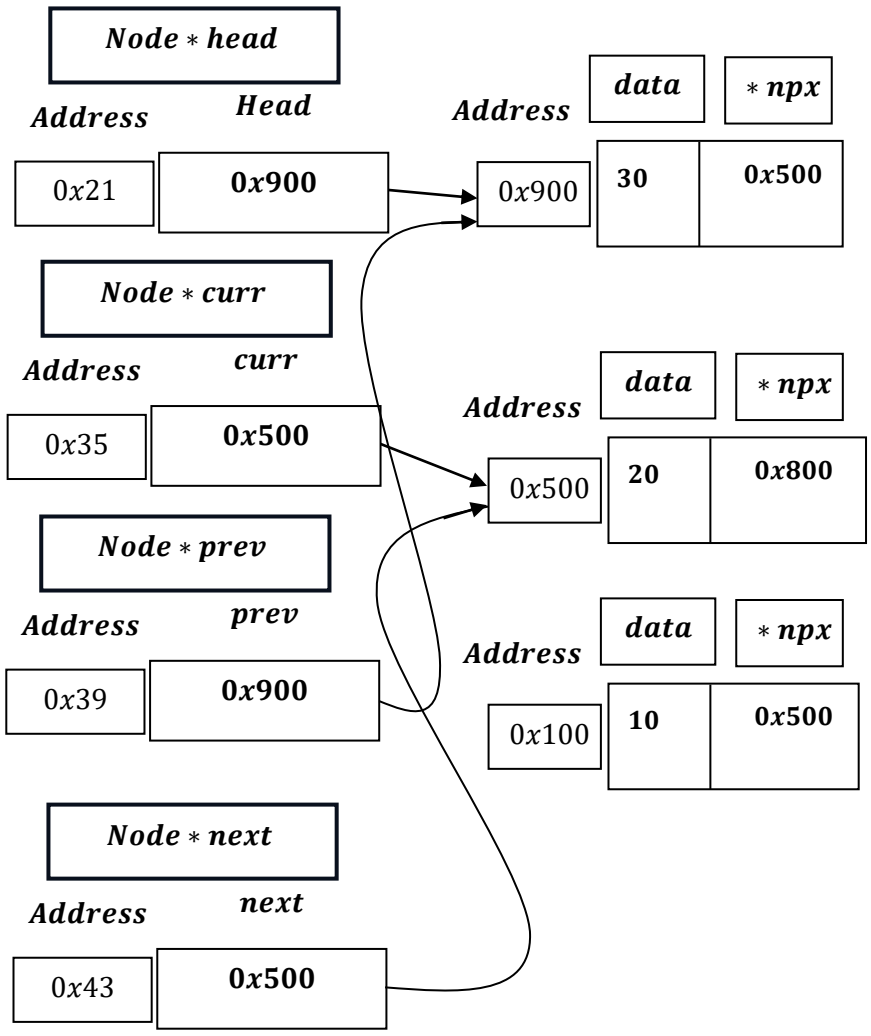
```
while (curr != nullptr)
{
    next = XOR(prev, curr->npx);
    if (next == nullptr){ break; }
    prev = curr;
    curr = next;
}
```

$\boldsymbol{curr = 0x900}$:

$\boldsymbol{next = XOR(prev, curr \rightarrow npx)}$; $\boldsymbol{\Longrightarrow XOR(0x000, 0x500) \Rightarrow 0x500}$;

$\boldsymbol{prev = curr = 0x900}$;

$\boldsymbol{curr = next = 0x500}$;



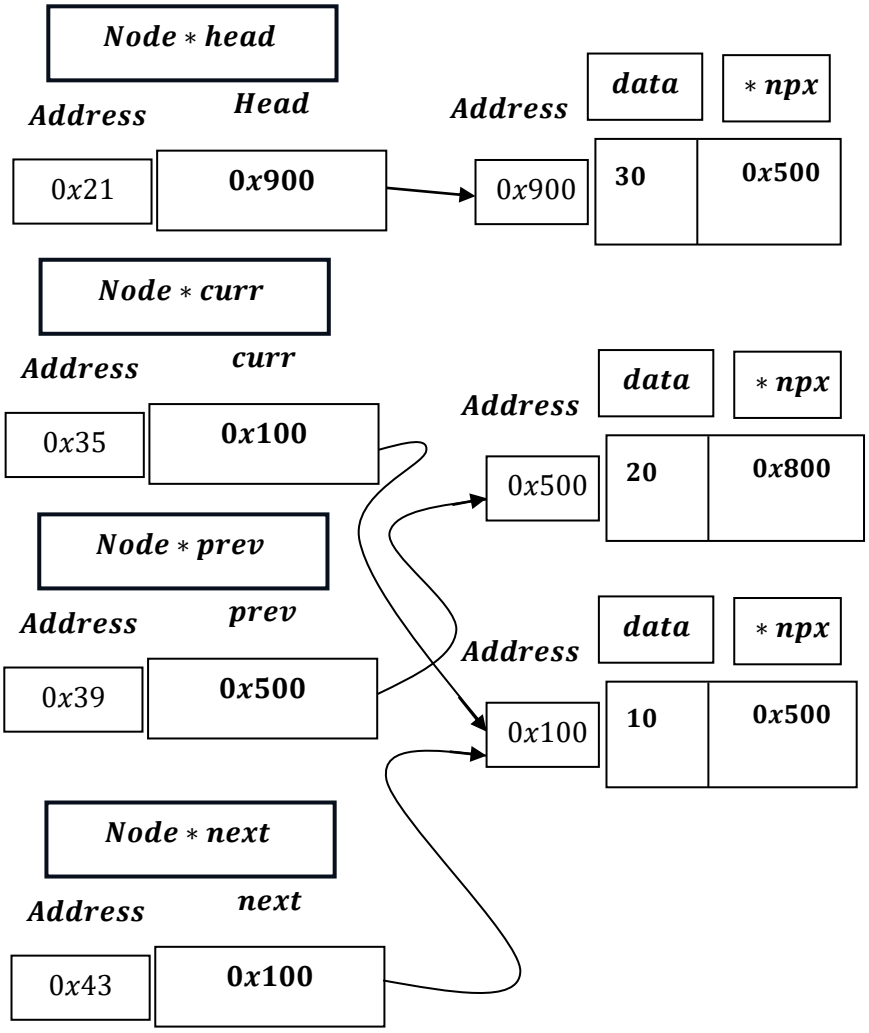
$\boldsymbol{curr = 0x500}$:

$\boldsymbol{next = XOR(prev, curr \rightarrow npx)}$; $\boldsymbol{\Longrightarrow XOR(0x900, 0x800)}$

$\boldsymbol{\Rightarrow 1001_2 \oplus 1000_2 = 0001_2 = 0x100}$;

$\boldsymbol{prev = curr = 0x500}$;

$\boldsymbol{curr = next = 0x100}$;



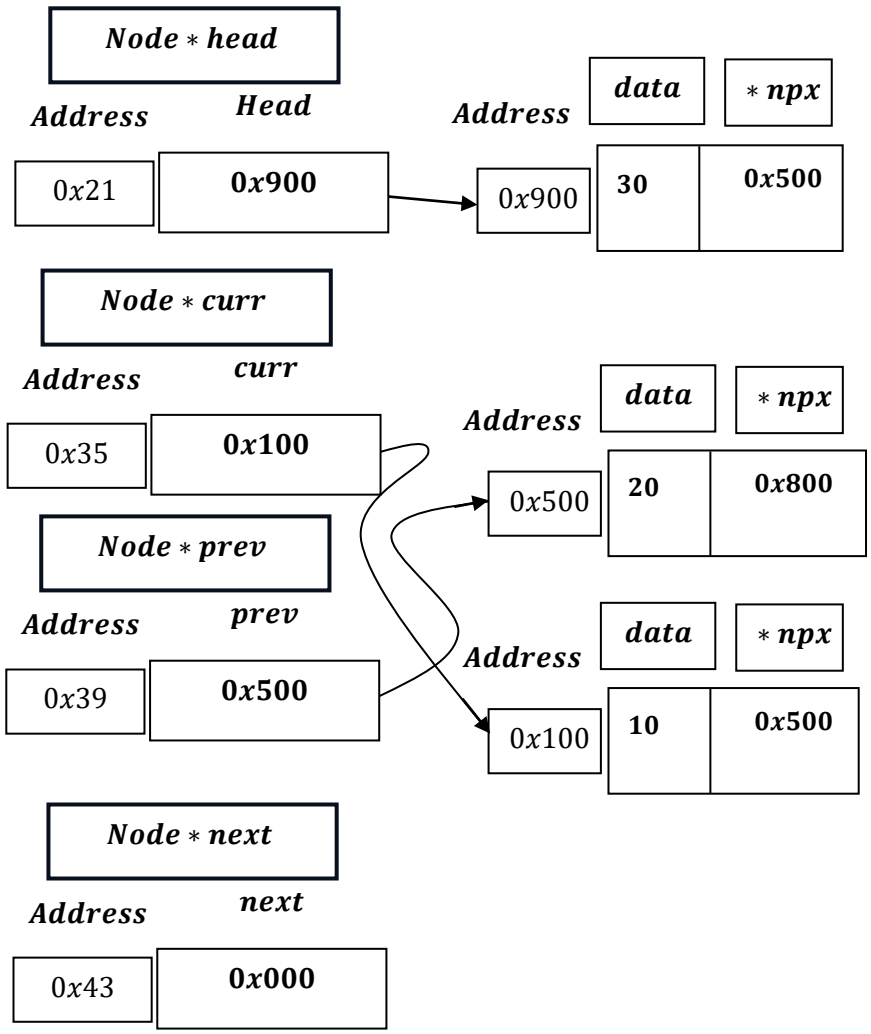
$curr = 0x100$:

$next = XOR(prev, curr \rightarrow npx)$; $\Longrightarrow XOR(0x500, 0x500)$

$\Rightarrow 0101_2 \oplus 0101_2 = 0000_2 = 0x000$;

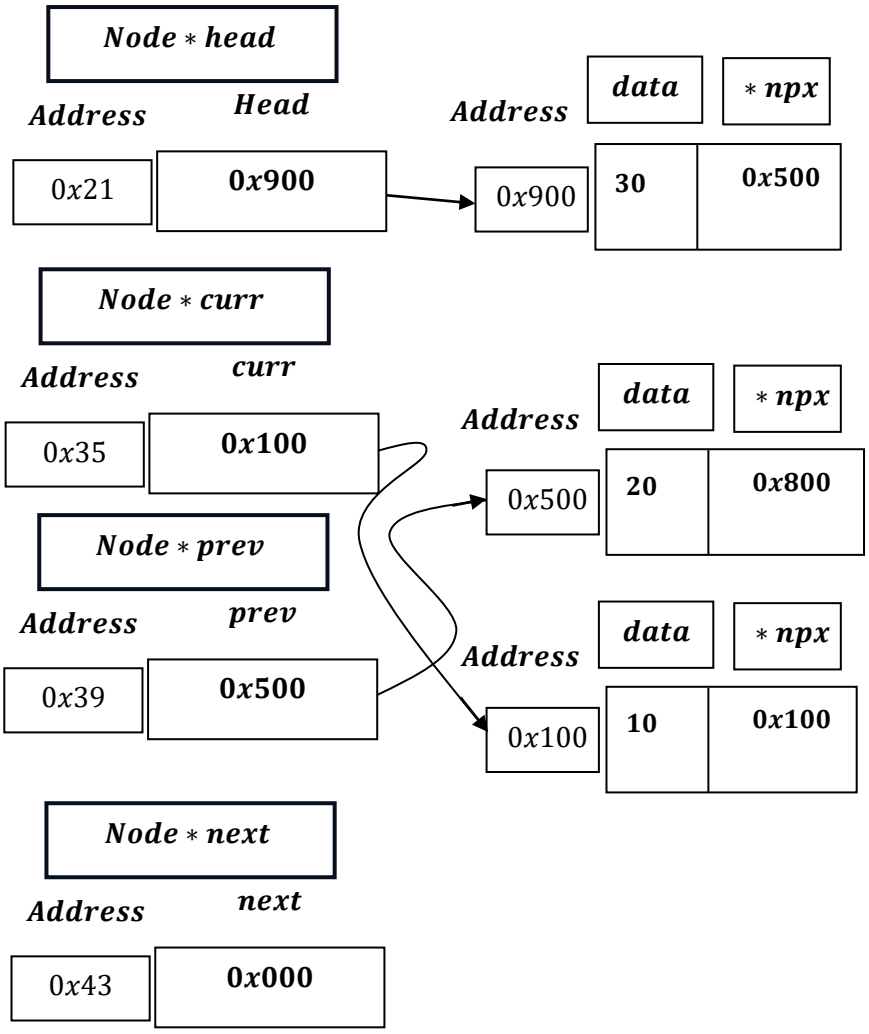
```
        if (next == nullptr){ break; }
```

As, break control flow statement used → it exits from the loop.



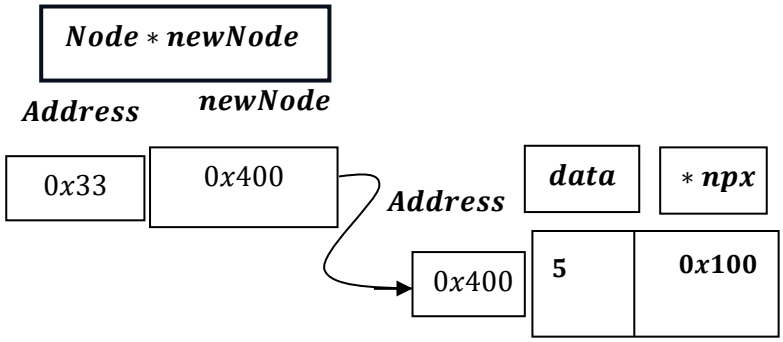

```
    curr->npx = XOR(prev, newNode);
```

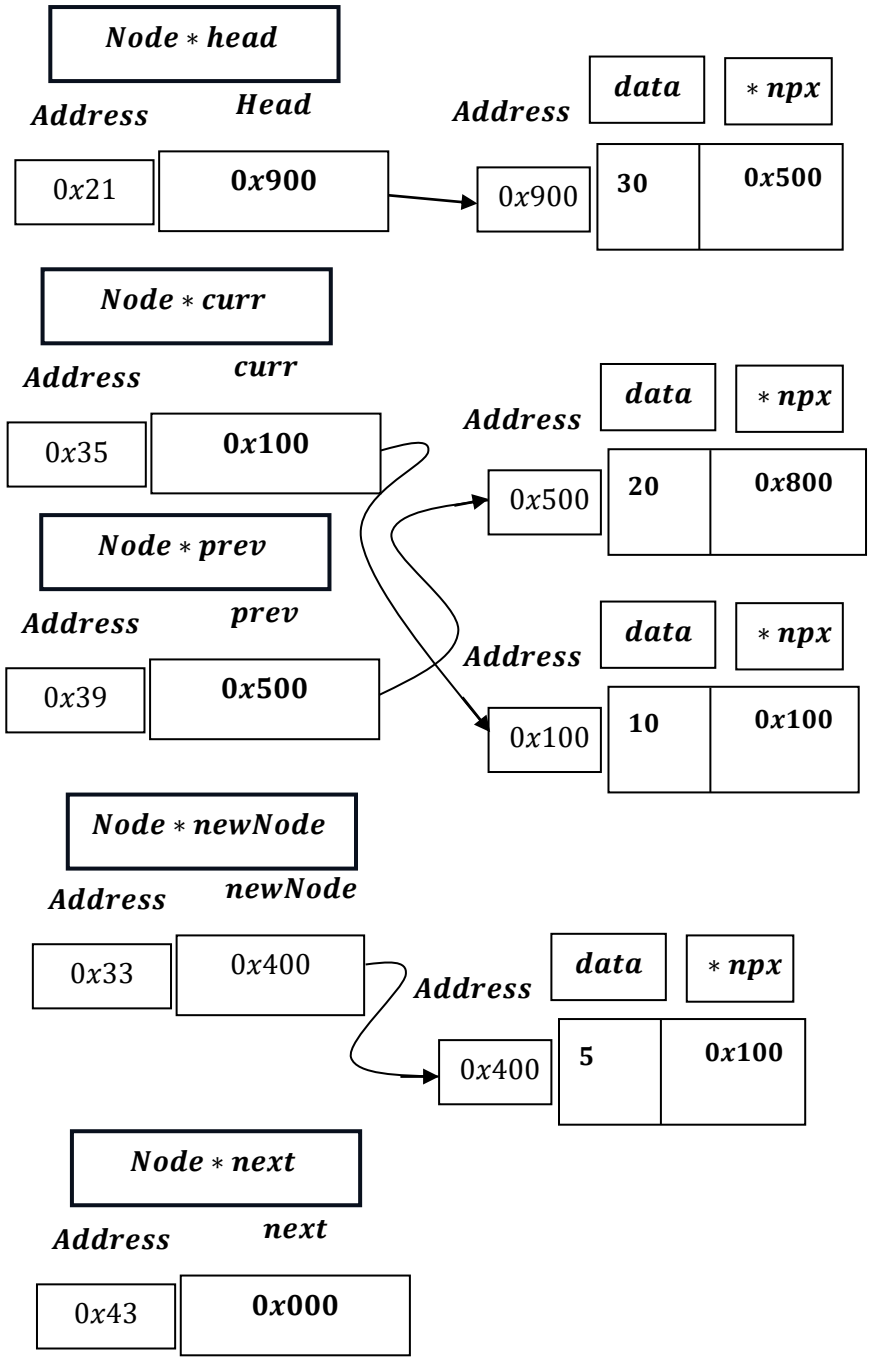
$curr \rightarrow npx = XOR(0x500, 0x400); = 0101_2 \oplus 0100_2 = 0001_2 = 0x100_2$.




```
        newNode->npx = XOR(curr, nullptr);
```

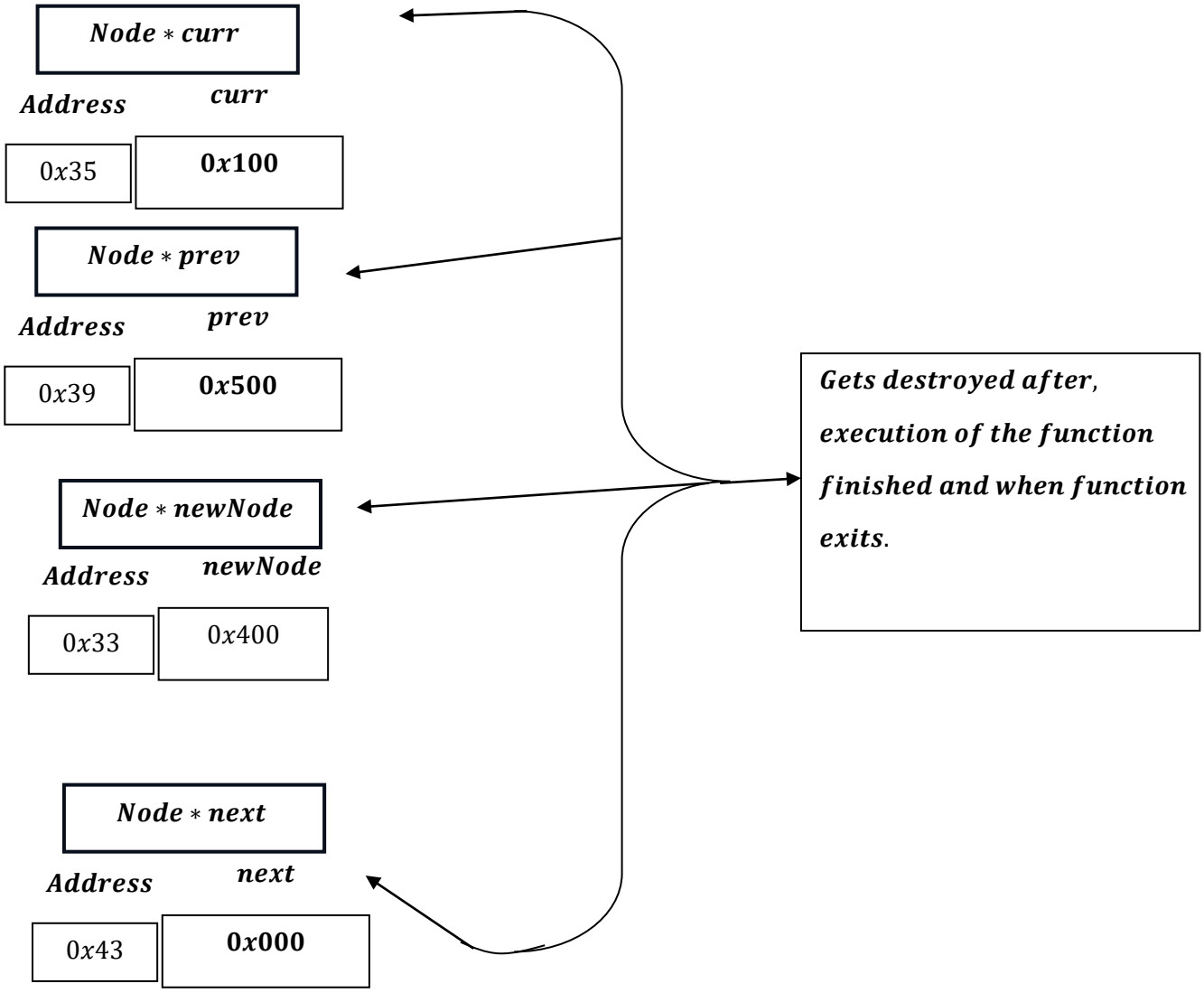
$$newNode \rightarrow npx = XOR(0x100, 0x000) = 0x100 \oplus 0x000 = 0x100.$$



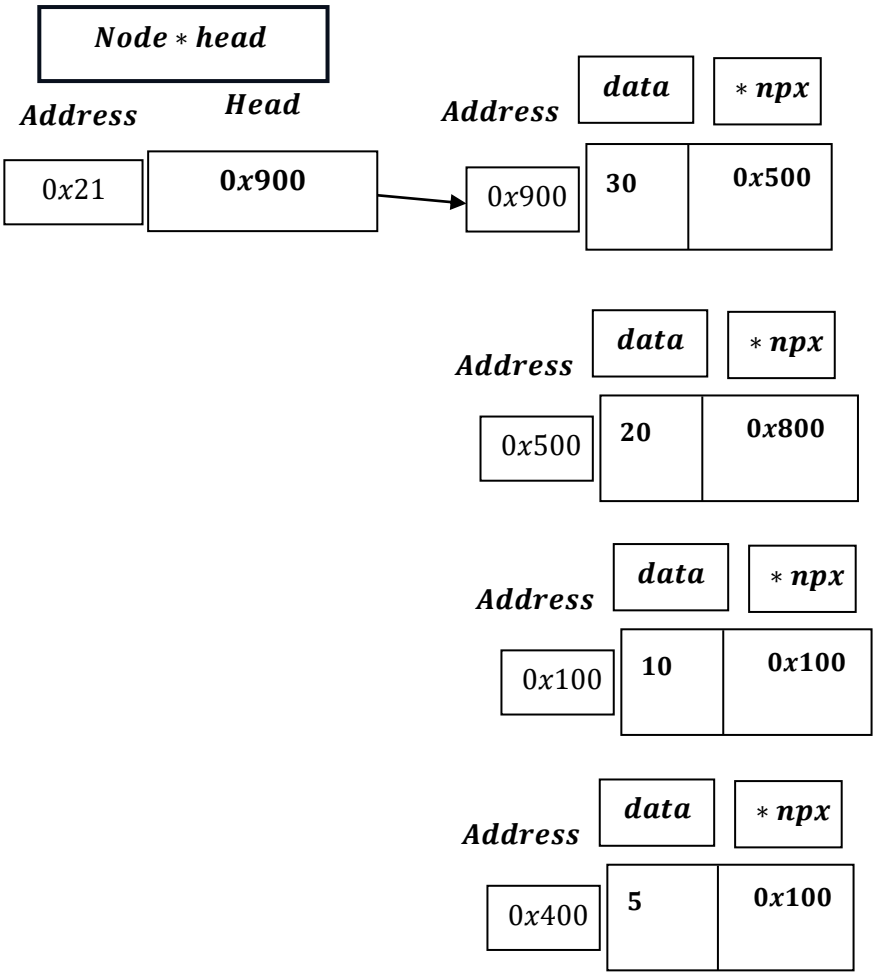


After Function ends , temporary pointer Node * newNode ,

Node * next, Node * prev, and Node * curr gets destroyed:



And we are left with:



Therefore, what we have observed, here:

***Node*: *A*(0*x*100)**

***data*: 10**

npx*: *NULL* ⊕ *HEAD

= 0*x*000 ⊕ 0*x*000

= 0*x*000

$Node: A(0x500)$

$data: 20$

$npx =$

$NULL \oplus B$

$= 0x000 \oplus 0x100$

$= 0x100$

$Node: B(0x100)$

$data: 10$

$npx =$

$(NULL \oplus B) \oplus A$

$(0x000 \oplus 0x000) \oplus 0x500$

$= 0x000 \oplus 0x500$

$= 0x500$

$Node: A(0x900)$

$data: 30$

$npx =$

$NULL \oplus B$

$= 0x000 \oplus 0x500$

$= 0x500$

$Node: B(0x500)$

$data: 20$

$npx =$

$(NULL \oplus A) \oplus C$

$= (0x000 \oplus 0x900) \oplus 0x100$

$= 0x900 \oplus 0x100$

$= 0x800$

$Node: C(0x100)$

$data: 10$

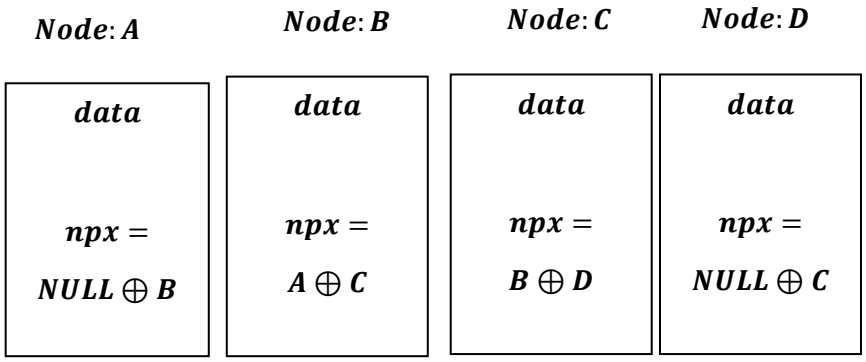
$npx =$

$NULL \oplus B$

$0x000 \oplus 0x500$

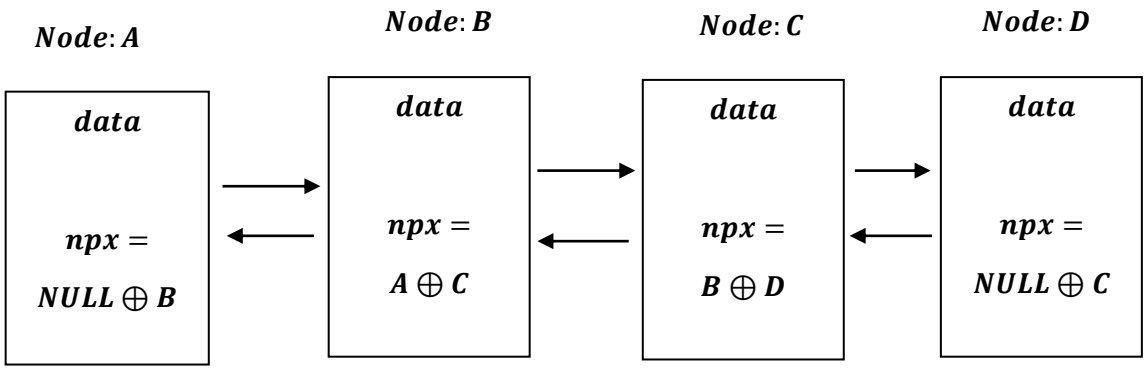
$= 0x500$

Similarly,



- ***There is no separate prev pointer and no separate next pointer stored.***
- ***The npx pointer by itself is not ``a pointer to the previous node`` and not ``a pointer to the next node``.***
- ***It's just a mixed value: the bitwise XOR of two addresses.***

Hence , by XORing only we can achieve:



- ***In C++, ``npx``, is not functioning as a pointer. It is functioning as a Storage Container.***
- ***We declare it as Node * npx only to tell the compiler: "Reserve 8 bytes of space here, because I plan to turn this numbers back into a pointer eventually."***
- ***It is effectively an Integer masquerading as a Pointer.***

The "Envelope" Analogy

Imagine a normal Linked List is a treasure hunt.

- ***Normal Pointer*: *You open an envelope, and inside is a slip of paper that says*: "*Go to House* #100." *You go there, and you find the house.***

- ***XOR Pointer*: *You open the envelope, and inside is a slip of paper that says*: "800".**

    - ***If you try to go to*** House #800, ***you will find an empty field or a cliff edge (Crash).***

    - ***Instead, you are supposed to take your current house number (100) and do math*: 800 *XOR* 100 = 900.**

"***Ah*! *The real destination is House* #900."

In a standard sense, the npx pointer violates the logical definition of a pointer.

- ***Standard Definition*: *A pointer holds the memory address of a specific, valid object.***

- ***XOR Definition*: *The npx variable holds a math result (a number) that does not point to any valid memory location.***

Why does the compiler allow this violation?

C++*: *It assumes you know what you are doing. It allows you to store any number in a pointer variable, as long as you cast it correctly. It only checks validity when you try to dereference (−> or ∗) it.

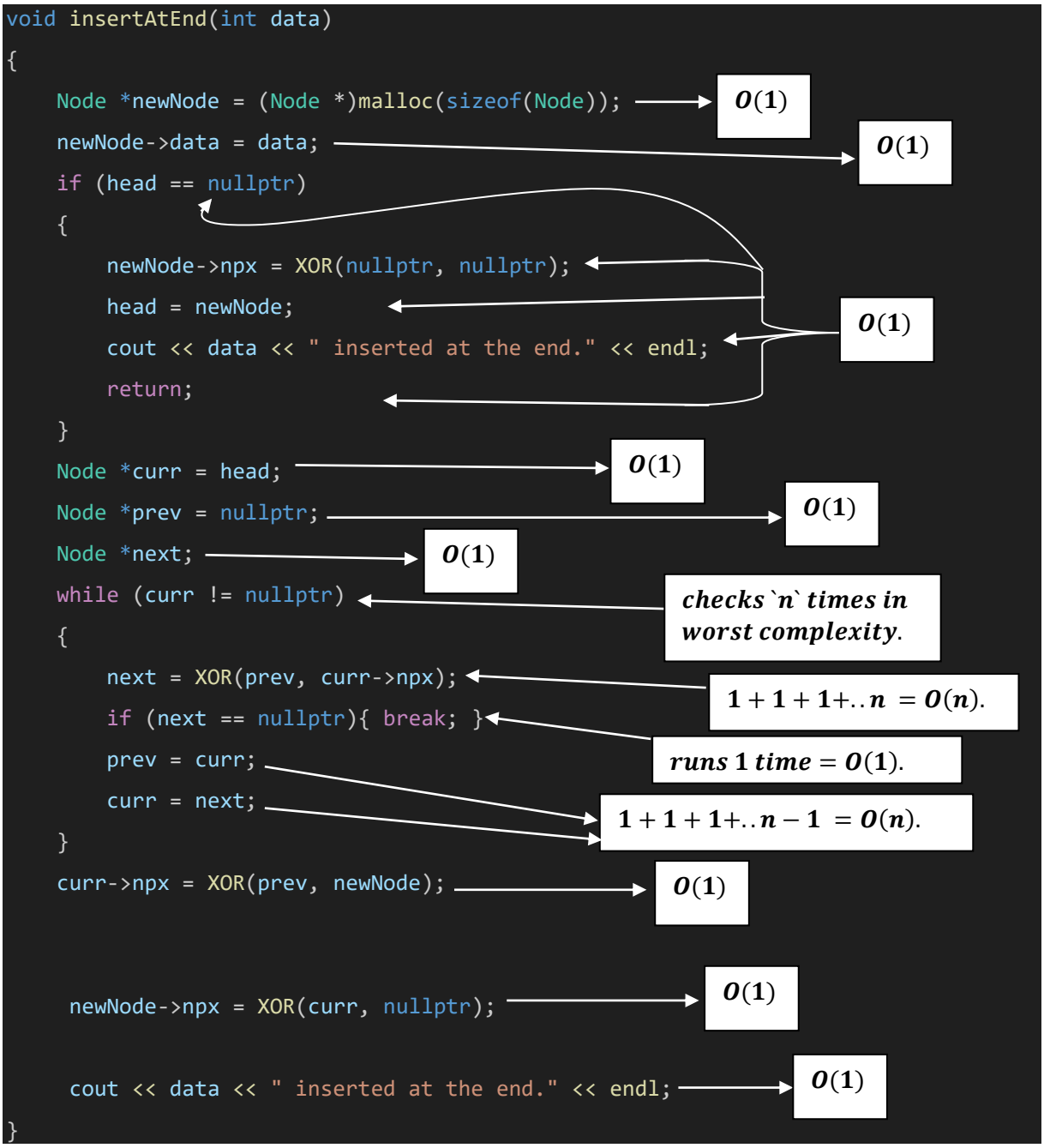
Summary:

- ***Does npx point to a memory address? No. It points to "garbage"*** (***mathematically calculated noise***).

- ***Does it violate pointer properties? Yes, it violates the property of "Dereferencability." You cannot look inside npx directly.***

- ***Why do we do it? To save space. We sacrifice the "safety" and "direct access" of a pointer to save 50% of our memory usage.***

Time Complexity Of The Program



- $\boldsymbol{Best\ Case\ is\ when\ ,List\ is\ empty, and\ single\ node\ is\ inserted} = \boldsymbol{\Omega(1)}$.

- $\boldsymbol{Worst\ Case\ when\ list\ is\ not\ empty\ especially\ the\ loop, traversing\ part}$:

  $= \boldsymbol{O(cn + \big(cn + c + (cn - c)\big) + (cn - c)\big), where\ c\ is\ constant}$
  $\boldsymbol{or, c\ \times O(n + \big(n + 1 + (n - 1)\big) + (n - 1)\big)}$
  $= \boldsymbol{O(n)}$.

- $\boldsymbol{Average\ Case\ ,there\ is\ no\ true\ average\ case\ as\ either\ it\ will\ be\ `c`(constant),\ when\ list\ is\ empty\ or\ traverse\ through\ list\ i.e.`n`\ times\ in\ worst\ complexity\ ,\ hence\ here\ Average\ Case = Worst\ Case = O(n)}$.

Memory Efficient Doubly Linked List	***Cases***	***Time Complexity***
InsertAt End	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****